

Sisteme de Operare

Cap 11. Stocarea

May 18, 2023

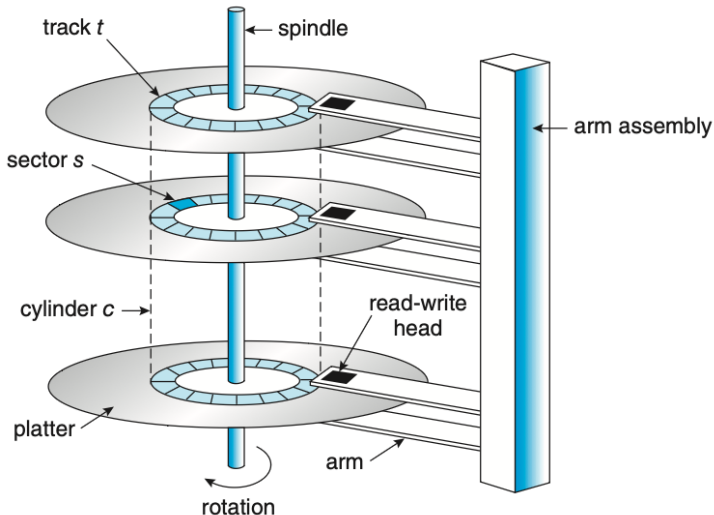
- 1 Structura sistemului de stocare
- 2 Planificatorul acceselor la HDD
- 3 Managementul dispozitivelor de stocare
- 4 Stocare suplimentară
- 5 Structuri RAID

- 1 Structura sistemului de stocare
- 2 Planificatorul acceselor la HDD
- 3 Managementul dispozitivelor de stocare
- 4 Stocare suplimentară
- 5 Structuri RAID

Dispozitive de tip HDD

- sistemul de stocare din calculator este memoria secundară, Non-Volatile Storage (NVS), adică memoria persistentă: Hard-Disk Drives (HDD) respectiv Non-Volatile Memory (NVM)
- memoria terțiară cuprinde dispozitive mai lente precum unitatea de bandă magnetică, CD/DVD/BluRay sau stocarea în cloud
- în principal ne referim la HDD și NVM (din care Solid-State Drive - SSD - face parte)
- într-un HDD, există mai multe discuri suprapuse pe același ax (se rotesc simultan)
- diametrele comune sunt de la 1.8 - 3.5 inchi (țoli)
- ambele suprafețe plane ale discului sunt acoperite cu un material magnetic
- informația se stochează prin detectarea tiparului magnetic
- cum discul se rotește, capul de citire care nu se mișcă descrie un **cilindru**; pot exista mii de cilindri concentrici

Structura de capete mobile ale unui HDD



Structura unui HDD

- cilindrul descrie pe fiecare față a unui disc o **pistă** (track); aceasta este compusă din segmente de cerc denumite **sectoare**
- fiecare sector are o mărime fixă și este cea mai mică unitate de transfer citită/scrisă o dată; mărimile comune sunt între 512 - 4096 bytes
- capacitatea de stocare comună a unui HDD este măsurată în GB sau TB
- motorul învârtă ansamblul de discuri cu viteze unghiulare între 60 - 250 [1/s]; vitezele comune sunt 5.400, 7.200, 10.000 sau 15.000 RPM (rotații pe minut); unele se pot opri (power down) când nu sunt folosite
- **rata de transfer** este volumul de date care circulă pe secundă între memorie și HDD; o metrică importantă este **timpul de acces**, ce are două componente: **seek time**, timpul necesar capului de a se poziționa deasupra cilindrului dorit și **rotational latency**, timpul necesar sectorului să ajungă sub capul de citire/scriere

Structura unui HDD (2)

- performanța se mărește prin adăugarea de memorie dinamică (DRAM) în controller-ul de HDD
- capul de citire/scriere "plutește" deasupra suprafeței discului, foarte aproape (de ordinul micronilor), pe o "pernă" de aer sau alt gaz precum heliul; când capul atinge accidental suprafața discului, defectul (**head crash**) nu poate fi reparat; datele sunt pierdute, de aceea ele trebuie protejate prin sisteme de back-up
- HDD-urile sunt sigilate, iar unele pot fi scoase din sistem în timp ce acesta este sub tensiune
- există unități media detașabile (removable), precum unitățile optice (CD, DVD, BluRay) sau memoriile de masă precum HDD-urile externe conectate pe USB
- de regulă datele publicate despre rata de transfer se referă la viteza cu care pot fi citați/scriși biții de pe mediul de stocare; aceasta este diferită de **rata de transfer efectivă**, viteza de livrare a blocurilor de date către SO

Dispozitive NVM

- sunt mai degrabă electrice, nu mecanice (nu au piese în mișcare)
- compuse dintr-un controller electronic și un semiconductor (circuit) NAND; mai fiabile
- figura prezintă un SSD, dar circuitele pot fi împachetate și sub formă de USB drive (stick)



Dispozitive NVM (2)

- folosite ca memorie secundară în laptop-uri pentru a le face mai eficiente și mai rapide
- deoarece ele pot fi mult mai rapide decât HDD-urile, dispozitivele NVM se conectează direct la magistrala sistemului (PCIe bus, Peripheral Component Interconnect express)
- sistemele NVM vin cu alte probleme de stocare și fiabilitate: ele pot fi scrise în pagini (similar cu abordarea de tip sector de la HDD), dar datele nu pot fi suprascrise; celulele trebuie mai întâi șterse, iar aceasta se face pe pagini care conțin câteva blocuri; operația de ștergere durează mult mai mult decât citirea (cea mai rapidă), sau scrierea (mai lentă decât citirea)
- fiind mai multe circuite, există mai multe căi de date, care funcționează în paralel; în timp ce se fac scrieri/citiri au loc ștergeri în altă parte

Dispozitive NVM (3)

- sistemele NAND se deteriorează cu fiecare ciclu de ștergere, iar după aproximativ 100,000 cicluri de ștergere, pagina nu mai poate reține date
- uzura determină durata de viață (**lifespan**), măsurată în **Drive Writes Per Day** (DWPD); de exemplu o memorie NAND de 1 TB cu 5 DWPD este de așteptat să suporte 5 TB scriși zilnic în perioada de garanție, fără eșec
- algoritmi de optimizare pentru accesele de scriere/citire sunt implementați de controller-ul NVM-ului, și sunt transparenți SO-ului

Algoritmi de management pentru NAND flash

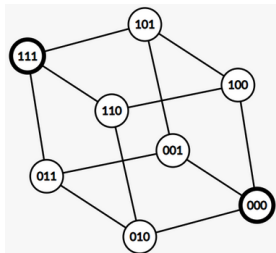
valid page	valid page	invalid page	invalid page
invalid page	valid page	invalid page	valid page

- la suprascrierea unui bloc al unui sistem de fișiere, pe NVM (flash) există și pagina veche, care conține acum date invalide, și pagina nouă, care conține datele noi
- evidența paginilor valide și invalide este ținută de controller într-un **Flash Translation Layer** (FTL); tabela mapează ce pagini fizice conțin blocuri logice valide
- o pagină conține mai multe blocuri; este posibil ca o pagină să conțină un mix de blocuri alocate respectiv nealocate sau blocuri invalide (scrise cu date vechi); de aceea ea nu poate fi încă stearsă

Overprovisioning

- există un mecanism de tip **garbage collection**, care copiază blocurile valide în blocuri libere din alte pagini, iar pagina este eliberată pentru a putea fi apoi ștearsă
- pentru ca garbage collector-ul să aibă unde să stocheze datele valide, se realizează un **overprovisioning**, se pune deoparte 20% din capacitate în acest scop
- sistemul de overprovisioning ajută și la reducerea uzurii; se caută ca toate paginile să fie șterse cu aceeași rată, pentru a evita ca unele să se uzeze mai repede decât altele
- datele sunt plasate în blocurile cu un număr de ștergeri cât mai mic
- ca și HDD-urile, NVM-urile folosesc **Error Correcting Codes (ECC)**; paginile care acumulează erori corectabile în mod repetat sunt marcate ca "bad" și nu vor mai fi folosite în scrierile următoare

Pe scurt despre codurile corectoare de erori (ECC)



- distanța Hamming este numărul de biți diferiți dintre două cuvinte de cod, i.e. între "0100" și "1101" distanța Hamming este 2
- un cod cu distanța Hamming d poate corecta cel mult $\lfloor \frac{d-1}{2} \rfloor$ erori¹
- folosite la HDD, NVM, chiar și la memoriile DRAM; mecanism transparent față de SO, implementat de controllerele de memorie

¹<https://www.section.io/engineering-education/understanding-error-correcting-codes-part-1/>

Memoria volatilă

- **drive-urile RAM** sunt create de drivere care folosesc o parte a RAM-ului pentru a o prezenta SO-ului ca un alt disk pe care poate scrie date; pe ele se creează de regulă sisteme de fișiere
- permite programatorului să plaseze date în memorie folosind primitivele de scris fișiere ale SO-ului
- pentru Linux avem `/dev/ram`, pe macOS comanda `diskutil` le creează
- la boot-are, sistemul Linux creează un sistem de fișiere temporar (`initrd`) ce permite altor servicii de sistem să aibă acces la root file system înainte ca alte componente ale sistemului de operare care lucrează cu dispozitivele de stocare să fie încărcate (modulele de lucru cu diverse sisteme de fișiere)

Metode de conectare a storage-ului secundar

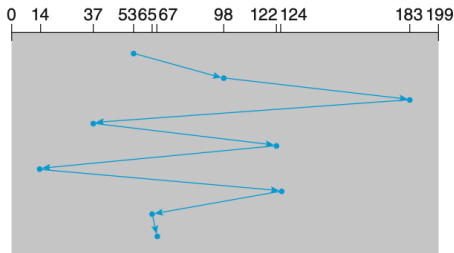
- folosind o magistrală (bus)
- Advanced Technology Attachment (ATA), Serial ATA (SATA), eSATA, Serial Attached SCSI (SAS), Universal Serial Bus (USB), Fiber Channel (FC)
- cea mai răspândită conexiune este SATA
- NVM-urile pot fi conectate și pe NVM express (NVMe), ce conectează dispozitivul la PCI-ul sistemului de calcul, crescând throughput-ul și scăzând latența; fiecare dispozitiv conține un **device controller**; **host controller-ul** este circuitul de la capătul magistralei (bus) din calculator
- CPU-ul plasează o comandă către host controller prin accesarea unor adrese mapate ca porturi I/O; host controller-ul trimite comenzi prin mesaje către device controller; transferul datelor se face de către host controller și device controller, prin DMA (Direct Memory Acces), fără intervenția CPU-ului; device controller-ul are un cache prin care se face transferul pe mediul de stocare

Maparea adreselor

- dispozitivele de stocare sunt văzute ca array-uri unidimensionale de blocuri logice
- fiecare block se mapează într-un sector sau bloc din NAND
- un Logical Block Address (LBA) este mai ușor de folosit de algoritmi decât conceptul de sector/cilindru/track sau chip/block/pagină
- de remarcat că nu fiecare track are același număr de sectoare; sau pot fi sectoare defecte (ascunse)
- track-urile de la periferie conține cu 40% mai multe sectoare decât cele de la centru
 - **constant linear velocity**: datele sunt stocate cu aceeași densitate, de aceea viteza de rotație va trebui să crească la parcurgerea cilindrilor centrali, ca să trimită date cu aceeași rată - abordare la CD, DVD
 - **constant angular velocity**: datele sunt mai dense la interior și mai "rare" la exterior - abordare pentru HDD

- 1 Structura sistemului de stocare
- 2 Planificatorul acceselor la HDD
- 3 Managementul dispozitivelor de stocare
- 4 Stocare suplimentară
- 5 Structuri RAID

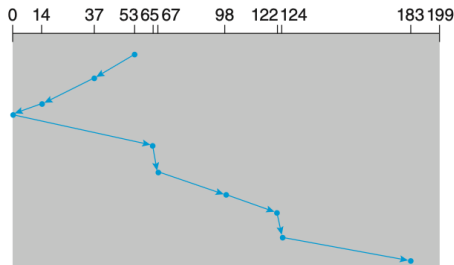
Planificatorul FCFS (First Come First Served)



- dispozitivele HDD moderne ascund configurația head/cilindru/track/sector, iar accesarea lor se face doar prin LBA (Logical Block Address); algoritmi folosesc ipoteza că adrese LBA apropiate necesită timpi mai mici de seek, accesul devine mai rapid
- există o coadă de cereri de blocuri la adresele specificate, ex. capul e la cilindrul 53 și vin cererile pentru cilindrii 98, 183, 37, 122, 14, 124, 65 și 67, "călătorind" 640 de cilindri²

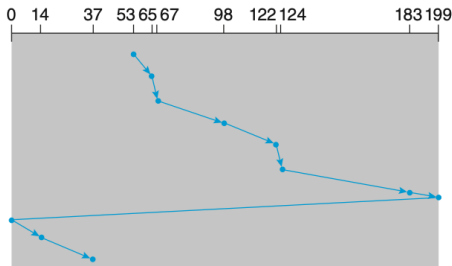
²Python: `sum(abs(x - y) for (x,y) in zip(l[:-1], l[1:]))`

Planificatorul SCAN



- capul pornește într-o direcție și o menține până la epuizare, după care parcurge cilindrii în direcția opusă; mai este denumit și **algoritmul liftului**

Planificatorul C-SCAN (Circular SCAN)



- variantă de SCAN
- observație: după ce capul a ajuns la o margine și o ia în sens invers, cererile din imediata vecinătate au fost deja satisfăcute; zona mai densă a cererilor este în cealaltă parte
- dacă există posibilitatea de a face rapid seek la 0, prin replanificare se poate accelera timpul total de servire, scade numărul de cilindri parcurși

- 1 Structura sistemului de stocare
- 2 Planificatorul acceselor la HDD
- 3 Managementul dispozitivelor de stocare**
- 4 Stocare suplimentară
- 5 Structuri RAID

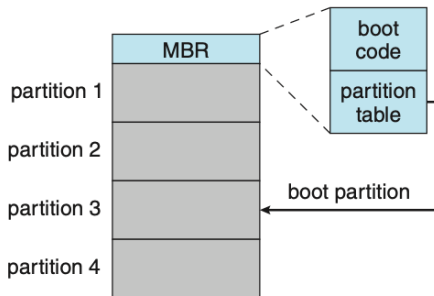
Formatarea, partiții, volume

- **low-level formatting**: scrierea structurilor de date speciale pentru accesul la date; acestea sunt folosite de controller pe parcursul operării (numărul de sector, de pagină, evidența defectelor, stocarea codurilor corectoare sau a Cyclic Redundancy Check - CRCurilor, sume de control care permit detecția erorilor)
- majoritatea drive-urilor sunt deja formate low-level din fabrică
- **partiția** este un ansamblu de blocuri contigue de pe dispozitivul de stocare (HDD, SSD, NVM), tratată de SO ca și când ar fi un dispozitiv individual
- SO va crea într-o partiție un sistem de fișiere; SO poate folosi mai multe sisteme de fișiere simultan
- "montarea" unui filesystem pune la dispoziția SO acea partiție
- în Linux fișierul `/etc/fstab` conține informații pentru SO de tip "ce partiție se montează în ce locație și folosind ce sistem de fișiere"

Formatarea, partiții, volume (2)

- al doilea pas este crearea unui **volum**; el este implicit când un filesystem se creează într-o singură partiție, dar preferăm ca un filesystem să fie împărțit pe mai multe partiții (ex. RAID)
- al treilea pas este **formatarea logică** - crearea structurilor sistemului de fișiere
- SO furnizează și posibilitatea de a accesa o partiție direct, ca array de blocuri, acces denumit **raw I/O**
- accesul direct la partiție sau la dispozitiv este folosit pentru alocarea **swap space**, folosit de SO pentru salvarea/restaurarea paginilor de memorie

Boot block



- **bootstrap** loader-ul, programul inițial de power-up, este stocat în memoria NVM firmware pe placa de bază (poate fi alterat și de viruși!); bootstrap loader-ul încarcă bootstrap-ul sistemului (ex. la Linux este grub2)
- un dispozitiv cu o partiție ce conține acest bootstrap program se numește **partiție de boot**; pt. Windows avem Master Boot Record, zona de la începutul drive-ului

- 1 Structura sistemului de stocare
- 2 Planificatorul acceselor la HDD
- 3 Managementul dispozitivelor de stocare
- 4 Stocare suplimentară**
- 5 Structuri RAID

Spații de stocare secundară

1 host-attached storage

- atașat la porturile I/O, de regulă SATA / USB
- cuprinde și Storage Area Network (SAN), rețea privată, date locale

2 network-attached storage (NAS)

- acces la stocare peste rețea, protocoale RPC precum NFS (UNIX) sau CIFS (Windows) peste TCP/IP, în LAN
- implementează sisteme de fișiere peste rețea (Network File System)
- iSCSI trimite blocuri de date peste rețea; SO "vede" un dispozitiv de la distanță ca pe un drive local

3 cloud storage

- folosește API-uri specifice, programele trebuie să folosească acel API și nu serviciile SO (ex. Amazon S3, Dropbox, MS OneDrive sau Apple iCloud)
- protocoalele NAS, NFS, CIFS se blochează dacă rețeaua are probleme în funcționare; sistemele cloud sunt proiectate tratând defectele rețelei ca pe o situație normală (regula, iar nu excepția)

- 1 Structura sistemului de stocare
- 2 Planificatorul acceselor la HDD
- 3 Managementul dispozitivelor de stocare
- 4 Stocare suplimentară
- 5 Structuri RAID**

Îmbunătățirea fiabilității prin redundanță

- Redundant Array of Independent Disks (RAID) este o modalitate de a organiza discurile atașate unui sistem pentru a-i îmbunătăți caracteristicile de performanță și fiabilitate
- fiecare disc are o metrică de fiabilitate, Mean Time Between Failures (MTBF)
- pentru MTBF de 100.000 ore, la un array de 100 discuri avem MTBF de $100.000 / 100 = 1.000$ ore, adică aprox. 42 de zile
- dacă stocăm o singură copie a datelor, va apărea fenomenul de pierdere a datelor, în mod repetat - inacceptabil
- cea mai simplă metodă de sporire a redundanței este **mirroring-ul**; fiecare disc logic constă din două discuri fizice, iar o scriere logică constă în două scrieri fizice pe fiecare drive fizic

Îmbunătățirea fiabilității prin redundanță (2)

- dacă presupunem independente defectările celor două drive-uri, și un timp **mediu de reparație** de 10 ore, atunci **timpul mediu până la pierderea datelor** este de $100.000 * 100.000 / (2 * 10) = 500 * 1.000.000$ ore, adică 57.000 ani
- în realitate căderile nu sunt independente (e.g. cataclisme, inundații, cutremure, respectiv învechirea simultană a drive-urilor)
- accesul la ansambluri de drive-uri montate în paralel îmbunătățește performanța; **bit-level striping** este modalitatea în care fiecare bit dintr-un byte este stocat pe un alt drive; se crește de 8 ori rata de transfer, sectoarele pe noul "drive" vor fi de 8 ori mai mari
- **block-level striping**-ul presupune repartizarea blocurilor unui volum pe mai multe discuri; pentru n drive-uri, blocul i este repartizat drive-ulul $i \bmod n + 1$

Nivele RAID



(a) RAID 0: non-redundant striping.

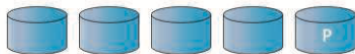


(b) RAID 1: mirrored disks.

- mirroring-ul are fiabilitate bună, dar este scump; striping-ul mărește rata de transfer, dar nu oferă fiabilitate
- nivelele RAID sunt scheme de folosire a striping-ului și biților de paritate³
- **RAID level 0:** array-uri de discuri cu striping la nivel de blocuri și fără redundanță
- **RAID level 1:** mirroring

³bitul de paritate este valoarea funcției XOR aplicată tuturor biților unui byte sau bloc de date

Nivele RAID (2)



(c) RAID 4: block-interleaved parity.



(d) RAID 5: block-interleaved distributed parity.

- **RAID level 4:** memory-style ECC; pentru N blocuri, se folosesc discurile $1 \dots N$ pentru a stoca blocurile $1 \dots N$, iar în discul $N+1$ se stochează blocul de paritate; dacă unul din discuri are defecte, știm exact unde apare acesta și putem recalcula acel bit folosind informația de paritate
- **RAID level 5:** distribuie informația de paritate între toate cele $N+1$ drive-uri; pentru 5 discuri, blocul n va avea informația de paritate pe discul $(n \bmod 5) + 1$; un bloc de paritate nu poate stoca paritatea asociată cu biții de pe același disc (scopul se ratează); împrăștierea parității echilibrează încărcarea; RAID 5 e cel mai folosit

Nivele RAID (3)

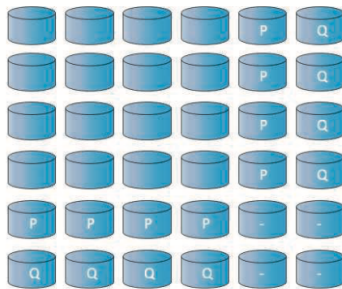


(e) RAID 6: P + Q redundancy.

- **RAID level 6:** denumit și **P + Q redundancy scheme**, seamănă cu RAID 5 dar stochează informație extra pentru a asigura împotriva căderilor multiple de drive-uri; nu se folosește funcția XOR în ambele blocuri de paritate (rezultate identice, al doilea este inutil) ci ECC-uri precum **grupurile Galois**⁴; se stochează 2 blocuri de date redundante la fiecare 4 blocuri de date, sunt tolerate două drive-uri defecte

⁴<https://people.computing.clemson.edu/~westall/851/rs-code.pdf>,
https://ro.wikipedia.org/wiki/Teoria_grupurilor#Teoria_lui_Galois, cubul Rubik este un grup de permutare

Nivele RAID (4)



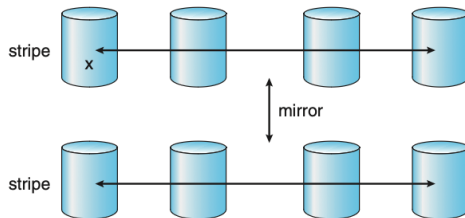
(f) Multidimensional RAID 6.

- RAID level 6 multidimensional:** pentru sute de drive-uri, RAID level 6 s-ar implementa cu multe drive-uri de date și doar 2 drive-uri de paritate; RAID 6 multidimensional organizează drive-urile într-o matrice și implementează RAID 6 atât pe linii cât și pe coloane

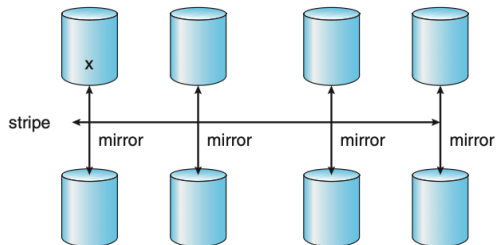
Nivele RAID (5)

- **RAID 0 + 1:** folosește atât RAID 0 (striping) cât și RAID 1 (mirroring); furnizează performanțe mai bune decât RAID 5, dar este mai scump, dublează numărul drive-urilor folosite; un set de drive-uri sunt striped, iar numărul lor este dublat pentru a implementa mirroring-ul
- **RAID 1 + 0:** drive-urile sunt oglindite în perechi iar perechile astfel formate sunt striped; dacă un singur drive pică în RAID 0 + 1, tot stripe-ul devine inaccesibil, lăsând disponibil doar celălalt stripe (mirrored); la RAID 1 + 0, dacă pică un drive, stripe-ul continuă să fie accesibil

RAID 0 + 1 și RAID 1 + 0



a) RAID 0 + 1 with a single disk failure.



b) RAID 1 + 0 with a single disk failure.

Implementări RAID

- în kernelul SO pentru managementul volumelor (ansambluri de partiții), sau în software-ul de sistem (servicii)
- în hardware-ul care guvernează magistrala Host-Bus Adapter (HBA); doar drive-urile ce folosesc magistrala HBA pot fi parte din RAID; soluția nu e flexibilă, dar ieftină (rapidă)
- în hardware-ul array-ului de stocare; array-ul "feliază" seturile RAID în volume mai mici, care sunt puse la dispoziția SO; array-urile pot fi parte dintr-un SAN (Storage Area Network)
- implementare în layer-ul de interconectare din SAN, ca drive-uri virtuale

Alegerea nivelului RAID

- timpul de rebuild cel mai rapid este RAID 1 (mirroring), respectiv ore pentru RAID 5
- RAID 0 este utilizat pentru aplicații unde pierderea datelor nu este critică, ex. calcule științifice unde datele pot fi refăcute din dataset-ul original stocat pe dispozitive externe
- RAID 1 este popular pentru fiabilitate ridicată cuplată cu timp de recuperare mic
- RAID 0 + 1 și 1 + 0 sunt utilizate unde și performanța și fiabilitatea sunt importante, ex. baze de date mici
- RAID 5 este preferat pentru stocarea volumelor moderate de date (RAID 1 face totuși risipă)
- RAID 6 și RAID 6 multidimensional sunt preferate pentru date mari, oferă performanță bună și protecție fără overhead prohibitiv